

DOCUMENTACIÓN TÉCNICA: DESARROLLO DE MICROSERVICIOS Y CAPA DE FRONTEND CON REFLEX (PYTHON FLEX)

- **Asignatura:** COM-600 Microservicios
 - **Tema de referencia:** Tema 1 — Fundamentos de la Arquitectura de Microservicios
 - **Tecnología analizada:** Reflex (anteriormente Pynecone), conocido coloquialmente como “Python Flex”
-

CONTEXTO ARQUITECTÓNICO: ARTICULACIÓN CON EL TEMA 1

1. Diferencia entre Monolito, Monolito Modular y Microservicios

Para comprender el rol de Reflex dentro de una arquitectura distribuida, es indispensable partir de la clasificación clásica de estilos arquitectónicos:

- **Monolito clásico:** Toda la lógica de negocio, la capa de presentación y el acceso a datos residen en un único proceso desplegable. El acoplamiento es máximo: un cambio en el módulo de facturación puede requerir recompilar y redespigar todo el sistema, incluyendo el módulo de inventario.
- **Monolito modular:** Introduce fronteras internas (módulos, paquetes, *bounded contexts* lógicos) dentro de un mismo proceso. Reduce el acoplamiento de código, pero **no elimina el acoplamiento de despliegue**: sigue existiendo un único artefacto ejecutable y, generalmente, una única base de datos compartida entre módulos.
- **Microservicios:** Cada servicio es un proceso independiente, con su propio ciclo de vida de despliegue, su propia base de datos (principio de *database-per-service*) y comunicación

exclusivamente a través de la red mediante contratos explícitos (REST, GraphQL, gRPC, mensajería asíncrona).

¿Dónde se sitúa Reflex? Reflex no es, por sí mismo, “un microservicio de lógica de negocio”. Es un **framework full-stack en Python que compila a una aplicación web** (frontend en Next.js/React más un backend en FastAPI/Starlette). Dentro de una arquitectura de microservicios correctamente diseñada, Reflex debe ocupar la posición de **capa de presentación desacoplada**, es decir, un servicio más del ecosistema —con su propio proceso y despliegue— cuya única responsabilidad es orquestar la experiencia de usuario y consumir, mediante llamadas de red, a los microservicios de dominio (Empleados, Nómina, Inventario, etc.).

2. Reflex como Backend for Frontend (BFF) o Microservicio de Interfaz

El patrón **Backend for Frontend (BFF)** propone que exista una capa intermedia, específica para cada tipo de cliente (web, móvil, etc.), cuya función es:

- Agregar y adaptar respuestas provenientes de múltiples microservicios de dominio.
- Manejar el estado de sesión/UI sin contaminar la lógica de negocio.
- Traducir contratos internos (gRPC, eventos) a formatos consumibles por el cliente (JSON sobre WebSocket/HTTP).

Reflex encaja naturalmente en este rol porque su modelo de ejecución ya separa un **backend** (que gestiona `rx.State` mediante FastAPI y WebSockets) de un **frontend compilado** (Next.js). Sin embargo, esto exige una disciplina arquitectónica estricta:

- **No debe centralizar la lógica de negocio.** Las reglas de dominio (cálculo de nómina, validaciones de negocio, políticas de descuentos, etc.) pertenecen a los microservicios correspondientes. Reflex únicamente orquesta llamadas y presenta resultados.
- **No debe compartir base de datos directa con otros microservicios.** Cada microservicio de dominio mantiene **propiedad exclusiva de sus datos** (*data ownership*). Reflex jamás

debe conectarse directamente al esquema de base de datos del microservicio de Empleados; toda interacción ocurre a través de la API pública de ese servicio (REST, GraphQL o gRPC). Esto preserva el encapsulamiento y permite que cada equipo evolucione su modelo de datos sin romper contratos externos.

3. Manejo de los Costos de Distribuir

Adoptar microservicios —y, por extensión, una capa Reflex desacoplada— no es gratuito. El Tema 1 enfatiza que se intercambia complejidad de código por complejidad operacional distribuida:

- **Latencia de red:** Cada llamada desde `rx.State` hacia un microservicio externo implica una travesía de red (serialización, transporte TCP/HTTP, deserialización). A diferencia de una llamada en memoria dentro de un monolito, esto introduce milisegundos —o más— que deben gestionarse con *timeouts*, *connection pooling* y llamadas asíncronas (`httpx.AsyncClient`).
- **Fallos parciales:** En un sistema distribuido, un microservicio puede estar caído mientras otros siguen operativos. Reflex, como orquestador de UI, debe implementar patrones de resiliencia: *circuit breakers*, reintentos con *backoff* exponencial, y estados de error explícitos en la interfaz (por ejemplo, mostrar “Servicio de nómina no disponible” en lugar de que la aplicación completa falle).
- **Consistencia eventual:** Al no existir una transacción ACID que abarque múltiples microservicios, los datos mostrados en la interfaz de Reflex pueden estar temporalmente desincronizados respecto al estado real de otro servicio (por ejemplo, tras una actualización propagada de forma asíncrona vía eventos). El diseño de la UI debe comunicar este hecho cuando sea relevante (indicadores de “sincronizando”, *timestamps* de última actualización, etc.).
- **Desacoplamiento:** A cambio de estos costos, se obtiene independencia de despliegue, escalabilidad granular (se puede escalar el microservicio de Empleados sin escalar Reflex) y

tolerancia a fallos localizada, evitando que un error en un dominio de negocio derribe toda la plataforma.

1. HISTORIA DEL LENGUAJE + FRAMEWORK Y LIBRERÍAS

1.1 Historia de Python

Python fue creado por **Guido van Rossum** y publicado por primera vez en 1991, con el objetivo de ofrecer una sintaxis legible y una curva de aprendizaje suave. A lo largo de sus versiones más relevantes para el desarrollo backend moderno destacan:

- **Python 3.0 (2008):** Ruptura de compatibilidad con Python 2 para modernizar el núcleo del lenguaje (manejo de cadenas Unicode por defecto, entre otros cambios).
- **Python 3.5 (2015):** Introducción de las palabras clave `async/await` (PEP 492), sentando las bases para la programación asíncrona nativa, esencial para frameworks web de alto rendimiento como FastAPI y, por extensión, Reflex.
- **Python 3.5+ (PEP 484):** Incorporación de **type hints** (anotaciones de tipo), que permiten declarar tipos estáticos opcionales (`def suma(a: int, b: int) -> int`). Este mecanismo es la base sobre la que se construye **Pydantic**, motor de validación de datos que utilizan tanto FastAPI como Reflex.
- **Python 3.6+ :** Soporte estable de `asyncio` como biblioteca estándar madura, habilitando servidores capaces de manejar miles de conexiones concurrentes (relevante para el modelo de WebSockets de Reflex).

Estas dos capacidades —tipado estático opcional y concurrencia asíncrona nativa— son el cimiento técnico que hace posible que un framework como Reflex pueda ofrecer reactividad en tiempo real y validación de datos robusta sin abandonar el lenguaje Python.

1.2 Evolución de Reflex (Python Flex)

- **Origen como Pynecone (2022):** El proyecto nació bajo el nombre *Pynecone*, fundado con la premisa de permitir construir aplicaciones web completas —frontend y backend— utilizando exclusivamente Python, eliminando la necesidad de que un equipo de backend aprenda JavaScript/TypeScript para construir interfaces.
- **Renombramiento a Reflex (2023):** El proyecto fue renombrado a **Reflex**, consolidando su identidad como framework “full-stack en Python puro” (*pure Python full-stack framework*).
- **Filosofía central:** El desarrollador escribe únicamente código Python; el *compiler* interno de Reflex traduce los componentes declarativos de la interfaz a una aplicación **Next.js/React** completamente funcional, mientras que la lógica de estado y eventos se ejecuta en un backend **FastAPI**. El resultado es una experiencia de desarrollo unificada, sin necesidad de mantener dos bases de código (una en Python, otra en JavaScript) ni dos equipos separados por tecnología.

1.3 Ecosistema de Librerías Internas

Componente	Rol dentro de Reflex
FastAPI	Expone el backend HTTP y gestiona el ciclo de vida del servidor Python que sirve el estado de la aplicación y los <i>endpoints</i> auxiliares.
Starlette	Framework ASGI de bajo nivel sobre el cual está construido FastAPI; provee el manejo de rutas asíncronas, middlewares y, crucialmente, el soporte de WebSockets que Reflex utiliza para sincronizar el estado del servidor con el cliente en tiempo real.

Componente	Rol dentro de Reflex
Pydantic	Motor de validación y serialización de datos basado en <i>type hints</i> . Reflex lo utiliza para definir y validar la forma de las clases de estado (<code>rx.State</code>), garantizando consistencia de tipos entre el backend y el frontend compilado.
Next.js	Framework de React utilizado como <i>target</i> de compilación. Reflex traduce los componentes Python a código React, y Next.js se encarga del <i>bundling</i> , <i>server-side rendering</i> opcional y optimización de assets estáticos.
React	Biblioteca de UI subyacente sobre la que corre el código generado; provee el modelo de componentes y el <i>virtual DOM</i> que finalmente renderiza el navegador.

2. VENTAJAS DE REFLEX

- **Desarrollo Full-Stack unificado sin necesidad de JavaScript/TypeScript:** Un mismo equipo, con un único lenguaje (Python), puede construir tanto la lógica de presentación como la lógica de orquestación del backend, reduciendo la fricción de contexto entre disciplinas y disminuyendo la curva de aprendizaje para equipos ya especializados en Python (científicos de datos, backend engineers).
- **Arquitectura reactiva sobre WebSockets en tiempo real:** Cada cambio en una variable de `rx.State` en el servidor se propaga automáticamente al cliente a través de una conexión

WebSocket persistente, sin que el desarrollador tenga que escribir manualmente lógica de *polling* o gestión manual de sockets. Esto habilita interfaces reactivas comparables a las de frameworks JavaScript modernos (React, Vue), pero gobernadas enteramente desde Python.

- **Compilación optimizada a Next.js para alto rendimiento:** Al compilar a Next.js, Reflex hereda las optimizaciones de dicho ecosistema: *code splitting*, generación de assets estáticos optimizados y un modelo de renderizado maduro y ampliamente adoptado en la industria.
- **Integración nativa con validación Pydantic y llamadas asíncronas (asyncio):** Los modelos de estado se benefician de la validación de tipos de Pydantic, reduciendo errores en tiempo de ejecución; adicionalmente, los manejadores de eventos (*event handlers*) pueden declararse como funciones `async def`, permitiendo realizar llamadas de red no bloqueantes hacia microservicios externos sin degradar la capacidad de respuesta de la interfaz.

3. CARACTERÍSTICAS PRINCIPALES

3.1 Estado del servidor (`rx.State`) y flujo unidireccional de datos

`rx.State` es una clase Python (subclase de `rx.State`) que centraliza todas las variables que determinan el contenido de la interfaz. El flujo de datos es **unidireccional**:

1. El usuario interactúa con la UI (clic, entrada de texto).
2. Se dispara un **evento**, que invoca un método del `State` en el servidor.
3. El método muta las variables del estado (o realiza operaciones asíncronas, como llamar a un microservicio).
4. El nuevo estado se serializa y se envía al cliente vía WebSocket.
5. La interfaz se re-renderiza automáticamente reflejando el nuevo estado.

Este patrón es conceptualmente equivalente al modelo *Flux/Redux* de React, pero gestionado íntegramente en Python del lado del servidor.

3.2 Componentes de interfaz (`rx.Component`)

Los elementos visuales se declaran de forma **declarativa** mediante funciones que retornan composiciones de `rx.Component` (por ejemplo, `rx.box`, `rx.text`, `rx.button`), análogos a los elementos HTML/JSS pero expresados como funciones Python. Estos componentes se enlazan directamente a variables del `State`, de modo que cualquier cambio en el estado se refleja automáticamente sin manipulación manual del DOM.

3.3 Manejo de eventos e integración con APIs externas

Los *event handlers* son métodos definidos dentro de una clase `State` que pueden:

- Modificar variables locales del estado.
- Ejecutar lógica asíncrona (`async def`) para consumir microservicios externos mediante clientes HTTP como `httplib`, o clientes gRPC generados desde definiciones Protobuf.
- Encadenar otros eventos o disparar notificaciones (`rx.toast`, redirecciones, etc.).

4. EJEMPLO REST CON EL RECURSO EMPLEADO + SWAGGER (OPENAPI)

A continuación, se presenta un microservicio REST completo, construido con **FastAPI** y **Pydantic**, que expone el recurso `Empleado` con operaciones CRUD y documentación interactiva automática.

```
"""
Microservicio REST – Gestión de Empleados
Framework: FastAPI + Pydantic
Puerto sugerido: 8001
Documentación interactiva (Swagger UI): http://localhost:8001/docs
```

```

Documentación alternativa (ReDoc): http://localhost:8001/redoc
"""

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, EmailStr, Field
from typing import List, Optional

#
-----
# 1. Configuración de la aplicación FastAPI con metadatos para
# OpenAPI/Swagger
#
-----
app = FastAPI(
    title="Microservicio de Empleados",
    description="API REST para la gestión del recurso Empleado dentro
    de la "
        "arquitectura de microservicios (COM-600).",
    version="1.0.0",
    docs_url="/docs",          # Swagger UI
    redoc_url="/redoc",        # ReDoc
)

#
-----
# 2. Modelo de datos: Empleado
#
-----
class Empleado(BaseModel):
    """Representa la entidad de dominio Empleado."""
    id: int = Field(..., description="Identificador único del
        empleado")
    nombre: str = Field(..., min_length=2, max_length=100)
    puesto: str = Field(..., description="Cargo o rol dentro de la
        organización")
    salario: float = Field(..., gt=0, description="Salario mensual en
        moneda local")
    email: EmailStr = Field(..., description="Correo corporativo del
        empleado")

    class Config:
        json_schema_extra = {
            "example": {
                "id": 1,
                "nombre": "Ana Torres",
                "puesto": "Ingeniera de Software",
                "salario": 8500.0,
            }
        }

```

```

        "email": "ana.torres@empresa.com",
    }
}

class EmpleadoCreate(BaseModel):
    """Modelo de entrada para la creación (no requiere id, se
    autogenera)."""
    nombre: str = Field(..., min_length=2, max_length=100)
    puesto: str
    salario: float = Field(..., gt=0)
    email: EmailStr

#
# -----
# 3. "Base de datos" en memoria (propiedad exclusiva de este
#    microservicio)
#
# -----
db_empleados: List[Empleado] = [
    Empleado(id=1, nombre="Ana Torres", puesto="Ingeniera de Software",
             salario=8500.0, email="ana.torres@empresa.com"),
    Empleado(id=2, nombre="Luis Fernández", puesto="Analista de Datos",
             salario=7200.0, email="luis.fernandez@empresa.com"),
]
_next_id = 3

#
# -----
# 4. Endpoints CRUD
#
# -----
@app.get("/empleados", response_model=List[Empleado],
         tags=["Empleados"])
async def listar_empleados():
    """Retorna la lista completa de empleados registrados."""
    return db_empleados

@app.get("/empleados/{empleado_id}", response_model=Empleado,
         tags=["Empleados"])
async def obtener_empleado(empleado_id: int):
    """Obtiene un empleado específico por su identificador."""
    for emp in db_empleados:
        if emp.id == empleado_id:

```

```

        return emp
    raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                        detail="Empleado no encontrado")

@app.post("/empleados", response_model=Empleado,
          status_code=status.HTTP_201_CREATED, tags=["Empleados"])
async def crear_empleado(payload: EmpleadoCreate):
    """Crea un nuevo empleado y lo agrega al almacén en memoria."""
    global _next_id
    nuevo = Empleado(id=_next_id, **payload.model_dump())
    db_empleados.append(nuevo)
    _next_id += 1
    return nuevo

@app.delete("/empleados/{empleado_id}",
            status_code=status.HTTP_204_NO_CONTENT,
            tags=["Empleados"])
async def eliminar_empleado(empleado_id: int):
    """Elimina un empleado existente por su identificador."""
    global db_empleados
    if not any(emp.id == empleado_id for emp in db_empleados):
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                            detail="Empleado no encontrado")
    db_empleados = [emp for emp in db_empleados if emp.id !=
                    empleado_id]
    return None

# Ejecución: uvicorn main:app --reload --port 8001

```

Documentación interactiva: Al levantar el servidor con `uvicorn main:app --reload --port 8001`, FastAPI genera automáticamente el esquema **OpenAPI** a partir de los *type hints* y modelos Pydantic. Este esquema se visualiza en `http://localhost:8001/docs` (interfaz Swagger UI), donde es posible explorar cada endpoint, ver los esquemas de petición/respuesta y ejecutar peticiones de prueba directamente desde el navegador.

5. EJEMPLO GRAPHQL CON EL RECURSO EMPLEADO

Implementación equivalente utilizando **Strawberry GraphQL**, integrada sobre FastAPI mediante GraphQLRouter.

```

"""
Microservicio GraphQL – Gestión de Empleados
Framework: Strawberry GraphQL + FastAPI
Puerto sugerido: 8002
Endpoint GraphQL (con GraphiQL habilitado):
    http://localhost:8002/graphql
"""

import strawberry
from strawberry.fastapi import GraphQLRouter
from fastapi import FastAPI
from typing import List, Optional

#
-----

# 1. Definición de tipos GraphQL
#
-----

@strawberry.type
class Empleado:
    id: int
    nombre: str
    puesto: str
    salario: float
    email: str

@strawberry.input
class EmpleadoInput:
    """Tipo de entrada utilizado por las mutations de creación."""
    nombre: str
    puesto: str
    salario: float
    email: str

#
-----

# 2. Almacén en memoria (propiedad exclusiva del microservicio GraphQL)
#
-----

```

```

_empleados_db: List[Empleado] = [
    Empleado(id=1, nombre="Ana Torres", puesto="Ingeniera de Software",
        salario=8500.0, email="ana.torres@empresa.com"),
    Empleado(id=2, nombre="Luis Fernández", puesto="Analista de Datos",
        salario=7200.0, email="luis.fernandez@empresa.com"),
]
_contador_id = 3

#
# -----
# 3. Queries
#
# -----
@strawberry.type
class Query:

    @strawberry.field
    def empleados(self) -> List[Empleado]:
        """Lista todos los empleados registrados."""
        return _empleados_db

    @strawberry.field
    def empleado(self, id: int) -> Optional[Empleado]:
        """Obtiene un empleado específico por id."""
        return next((e for e in _empleados_db if e.id == id), None)

#
# -----
# 4. Mutations
#
# -----
@strawberry.type
class Mutation:

    @strawberry.mutation
    def crear_empleado(self, datos: EmpleadoInput) -> Empleado:
        """Crea un nuevo empleado a partir del input recibido."""
        global _contador_id
        nuevo = Empleado(
            id=_contador_id,
            nombre=datos.nombre,
            puesto=datos.puesto,
            salario=datos.salario,
            email=datos.email,
        )

```

```

        _empleados_db.append(nuevo)
        _contador_id += 1
        return nuevo

#
# -----
# 5. Ensamblado del esquema e integración con FastAPI
#
# -----

schema = strawberry.Schema(query=Query, mutation=Mutation)
graphql_app = GraphQLRouter(schema)

app = FastAPI(title="Microservicio GraphQL de Empleados")
app.include_router(graphql_app, prefix="/graphql")

# Ejecución: uvicorn main:app --reload --port 8002
# Ejemplo de query:
#   query { empleados { id nombre puesto } }
# Ejemplo de mutation:
#   mutation {
#     crearEmpleado(datos: {nombre: "Marta Ruiz", puesto: "QA",
#       salario: 6000, email: "marta@empresa.com"}) {
#       id nombre
#     }
#   }

```

6. EJEMPLO gRPC CON EL RECURSO EMPLEADO

6.1 Archivo de contrato Protobuf (empleado.proto)

```

syntax = "proto3";

package empleados;

// Servicio gRPC para la gestión del recurso Empleado
service EmpleadoService {
  rpc ListarEmpleados (EmpleadoListaRequest) returns
    (EmpleadoListaResponse);
  rpc ObtenerEmpleado (EmpleadoIdRequest) returns (EmpleadoResponse);
  rpc CrearEmpleado (CrearEmpleadoRequest) returns (EmpleadoResponse);
}

```

```

}

// Mensaje de entidad Empleado
message Empleado {
    int32 id = 1;
    string nombre = 2;
    string puesto = 3;
    double salario = 4;
    string email = 5;
}

// --- Peticiones y respuestas ---

message EmpleadoListaRequest {}

message EmpleadoListaResponse {
    repeated Empleado empleados = 1;
}

message EmpleadoIdRequest {
    int32 id = 1;
}

message EmpleadoResponse {
    Empleado empleado = 1;
}

message CrearEmpleadoRequest {
    string nombre = 1;
    string puesto = 2;
    double salario = 3;
    string email = 4;
}

```

Compilación del contrato (genera empleado_pb2.py y empleado_pb2_grpc.py):

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=.
empleado.proto
```

6.2 Servidor gRPC en Python (server.py)

```

"""
Microservicio gRPC – Gestión de Empleados

```

```

Puerto sugerido: 50051
Requiere: grpcio, grpcio-tools (contrato compilado previamente)
"""

import grpc
from concurrent import futures

import empleado_pb2
import empleado_pb2_grpc

#
-----
# 1. Almacén en memoria (propiedad exclusiva del microservicio gRPC)
#
-----
_empleados_db = [
    {"id": 1, "nombre": "Ana Torres", "puesto": "Ingeniera de
Software",
    "salario": 8500.0, "email": "ana.torres@empresa.com"},
    {"id": 2, "nombre": "Luis Fernández", "puesto": "Analista de
Datos",
    "salario": 7200.0, "email": "luis.fernandez@empresa.com"},
]
_next_id = 3

def _dict_a_mensaje(emp: dict) -> empleado_pb2.Empleado:
    """Convierte un diccionario interno al mensaje Protobuf
Empleado."""
    return empleado_pb2.Empleado(
        id=emp["id"], nombre=emp["nombre"], puesto=emp["puesto"],
        salario=emp["salario"], email=emp["email"],
    )

#
-----
# 2. Implementación del Servicer definido en el contrato .proto
#
-----
class EmpleadoServicer(empleado_pb2_grpc.EmpleadoServiceServicer):

    def ListarEmpleados(self, request, context):
        """Retorna la lista completa de empleados como mensajes
Protobuf."""
        mensajes = [_dict_a_mensaje(e) for e in _empleados_db]

```

```

        return empleado_pb2.EmpleadoListaResponse(empleados=mensajes)

def ObtenerEmpleado(self, request, context):
    """Busca un empleado por id; si no existe, retorna error
    NOT_FOUND."""
    emp = next((e for e in _empleados_db if e["id"] == request.id),
    None)
    if emp is None:
        context.set_code(grpc.StatusCode.NOT_FOUND)
        context.set_details("Empleado no encontrado")
        return empleado_pb2.EmpleadoResponse()
    return empleado_pb2.EmpleadoResponse(empleado=
        _dict_a_mensaje(emp))

def CrearEmpleado(self, request, context):
    """Crea un nuevo empleado a partir de la petición recibida."""
    global _next_id
    nuevo = {
        "id": _next_id, "nombre": request.nombre, "puesto":
        request.puesto,
        "salario": request.salario, "email": request.email,
    }
    _empleados_db.append(nuevo)
    _next_id += 1
    return empleado_pb2.EmpleadoResponse(empleado=
        _dict_a_mensaje(nuevo))

#
# -----
# 3. Arranque del servidor gRPC en el puerto TCP 50051
#
# -----

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    empleado_pb2_grpc.add_EmpleadoServiceServicer_to_server(
        EmpleadoServicer(), server
    )
    server.add_insecure_port("[::]:50051")
    server.start()
    print("Servidor gRPC de Empleados escuchando en el puerto
        50051...")
    server.wait_for_termination()

if __name__ == "__main__":
    serve()

```

7. CONSUMO E INTEGRACIÓN DE MICROSERVICIOS DESDE REFLEX

El siguiente ejemplo ilustra una aplicación **Reflex** cuyo `rx.State` consume, de forma asíncrona y mediante `httpx`, el microservicio REST de Empleados descrito en la Sección 4. Este código evidencia el rol de Reflex como **capa de presentación desacoplada (BFF)**: no accede a ninguna base de datos directamente, únicamente orquesta llamadas HTTP hacia el microservicio de dominio.

```
"""
Aplicación Reflex – Frontend de consulta de Empleados
Consume el microservicio REST de Empleados (Sección 4) vía httpx.
Ejecución: reflex run
"""

import httpx
import reflex as rx
from typing import List

# URL del microservicio REST de Empleados (podría provenir de variables
# de entorno)
EMPLEADOS_SERVICE_URL = "http://localhost:8001/empleados"

class EmpleadoUI(rx.Base):
    """Modelo liviano utilizado únicamente para tipar los datos en el
    frontend."""
    id: int
    nombre: str
    puesto: str
    salario: float
    email: str

class EmpleadosState(rx.State):
    """Estado del servidor: orquesta la comunicación con el
    microservicio."""
```

```

empleados: List[EmpleadoUI] = []
cargando: bool = False
error: str = ""

async def cargar_empleados(self):
    """
        Evento asíncrono que consulta el microservicio REST de
        Empleados.
        Implementa manejo básico de fallos parciales (timeout /
        servicio caído),
        coherente con los costos de distribuir discutidos en el Tema 1.
    """
    self.cargando = True
    self.error = ""
    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            respuesta = await client.get(EMPLEADOS_SERVICE_URL)
            respuesta.raise_for_status()
            datos = respuesta.json()
            self.empleados = [EmpleadoUI(**item) for item in datos]
    except httpx.TimeoutException:
        self.error = "El microservicio de Empleados no respondió a
            tiempo."
    except httpx.HTTPStatusError as exc:
        self.error = f"Error del microservicio:
            {exc.response.status_code}"
    except httpx.RequestError:
        self.error = "No fue posible conectar con el microservicio
            de Empleados."
    finally:
        self.cargando = False

def fila_empleado(empleado: EmpleadoUI) -> rx.Component:
    """Renderiza una fila de la tabla para un empleado individual."""
    return rx.table.row(
        rx.table.cell(empleado.id),
        rx.table.cell(empleado.nombre),
        rx.table.cell(empleado.puesto),
        rx.table.cell(f" {empleado.salario}"),
        rx.table.cell(empleado.email),
    )

def index() -> rx.Component:
    """Página principal: lista dinámica de empleados renderizada desde
        el estado."""

```

```

return rx.container(
    rx.heading("Directorio de Empleados", size="7",
margin_bottom="1em"),
    rx.button(
        "Cargar empleados",
        on_click=EmpleadosState.cargar_empleados,
        loading=EmpleadosState.cargando,
    ),
    rx.cond(
        EmpleadosState.error != "",
        rx.callout(EmpleadosState.error, color_scheme="red",
margin_top="1em"),
    ),
    rx.table.root(
        rx.table.header(
            rx.table.row(
                rx.table.column_header_cell("ID"),
                rx.table.column_header_cell("Nombre"),
                rx.table.column_header_cell("Puesto"),
                rx.table.column_header_cell("Salario"),
                rx.table.column_header_cell("Email"),
            )
        ),
        rx.table.body(
            rx.foreach(EmpleadosState.empleados, fila_empleado)
        ),
        margin_top="1em",
    ),
    padding="2em",
)

#
-----
# Registro de la aplicación Reflex
#
-----
app = rx.App()
app.add_page(index, on_load=EmpleadosState.cargar_empleados)

```

En este ejemplo se observan de manera concreta los principios expuestos en el contexto arquitectónico:

- **Desacoplamiento:** EmpleadosState no conoce el esquema de base de datos del microservicio de Empleados; únicamente conoce su contrato HTTP público.

- **Manejo de fallos parciales:** Se capturan explícitamente los escenarios de *timeout*, error HTTP y error de conexión, evitando que una falla del microservicio de dominio derribe toda la interfaz.
- **Asincronía:** El uso de `httpx.AsyncClient` dentro de un *event handler async* def permite que el servidor Reflex continúe atendiendo otras conexiones WebSocket mientras espera la respuesta del microservicio.